



TypeConversions

🕒 Created time	July 11, 2026 10:31 AM
☰ Category	CommonVariables
👤 Last edited by	 R.A R.A
🕒 Last updated time	July 11, 2026 1:56 PM

```
Console.WriteLine("What's your age: ");
string? ageText = Console.ReadLine();

//Console.ReadLine(ageText + 15);

//int age = int.Parse(ageText);

//int age;

bool isValidInt = int.TryParse(ageText, out int age);

Console.WriteLine($"This is valid: {isValidInt}. The number was {age}.");

Console.WriteLine(age + 15);

double testDouble = age;
decimal testDecimal = (decimal)testDouble;
```

ההבדל בין

Console.Write

לבין

Console.WriteLine

הוא ש:

Console.Write

לא מוסיף שורה חדשה (מעבר שורה) בסוף הטקסט.

הפקודה Console.WriteLine

פקודה זו מדפיסה את הטקסט למסך הקונסולה ומיד לאחר מכן מבצעת ירידת שורה אוטומטית (כמו לחיצה על מקש Enter).

המשמעות היא שכל טקסט או קלט שיגיע מיד לאחר מכן יופיע בשורה החדשה שמתחת. משתמשים בפקודה זו לרוב עבור הודעות מידע, כשאנחנו רוצים להפריד תכולה לשורות נפרדות וברורות.

הפקודה Console.Write

פקודה זו מדפיסה את הטקסט למסך הקונסולה אך משאירה את הסמן באותה השורה בדיוק, מיד בסוף התו האחרון שהודפס.

לא מתבצע מעבר שורה אוטומטי, ולכן כל מה שיקרה לאחר מכן יתבצע בהמשך ישיר של אותה השורה.

הצגנו למשתמש את השאלה לגבי הגיל שלו.

השימוש ב-**Console.Write** מבטיח שכאשר המשתמש יתחיל להקליד את התשובה שלו באמצעות הפקודה **Console.ReadLine**, ההקלדה תתבצע בהמשך ישיר לאותה שורה שבה הוצגה השאלה, ולא בשורה מתחת. שיטה זו יוצרת חווית משתמש טבעית ונוחה יותר בעת הזנת נתונים בקונסולה, ומדמה מילוי של שדה בטופס.

~~~~~

## הסוגים הנפוצים ביותר להמרה וקבלת קלט מהמשתמש:

סוגי המשתנים שנרצה להמיר:

הנפוץ ביותר הוא הרעיון של המרה מ-**String** למשהו אחר.

לעתים קרובות אנו מקבלים את הערכים שלנו מה-**Console** או ממקור אחר, ואנחנו מתייחסים אליהם כמחרוזות. **לדוגמה:** נכתוב בקונסול את השאלה

```
"What's your age: "
```

ואנו מצפים שהמשתמש יקליד את התשובה.

כדי ללכוד את המידע הזה נשתמש בפקודה:

```
Console.ReadLine();
```

פקודה זו תלכוד את כל מה שהמשתמש מקליד עד שהוא לוחץ על **Enter**.

השתמשנו בזה בעבר, ונוכל להשתמש בזה כדי ללכוד כל מה שיוקלד ולשמור את זה בתוך משתנה מסוג מחרוזת שיכולה להיות ריקה

```
Nullable string?
```

נוכל להגדיר את המשתנה בנפרד ואז לקלוט אליו, או להגדיר אותו בשורה אחת (**inline**).  
שתי הדרכים מצוינות.  
קראנו למשתנה:

```
ageText
```

מכיוון שהמידע מגיע כמחרוזת טקסט.

## הבעיה בחיבור מחרוזות לעומת חיבור מספרים:

אם נרצה לגלות בן כמה יהיה האדם בעוד 15 שנים, אנחנו לא יכולים פשוט לכתוב:

```
Console.WriteLine(ageText + 15);
```

למרות שהקומפיילר לא נותן לנו אזהרה, אם נריץ את זה, הקונסול ישאל את המשתמש לגילו, המשתמש יקליד **43**, ילחץ **Enter**, והקומפיילר יכתוב שבעוד 15 שנה הוא יהיה בן **4315**.  
מה שקרה כאן הוא שאפשר להשתמש בסימן הפלוס לא רק לחיבור מספרים, אלא גם לחיבור מחרוזות.  
לכן, מה שהתוכנה עשתה זה לקחת את המספר והתייחסה אליו כמחרוזת.  
כשכתבנו פלוס, התכווננו להתייחס ל-15 כמחרוזת, ואז לצרף את המחרוזת "15" אל המחרוזת **ageText** שלנו, מה שיוצר את התוצאה "4315".  
זו לא התוצאה שאנחנו מחפשים, זה לא יעבוד וזה לא קוד טוב.

## המרה בסיסית באמצעות `int.Parse`

אז איך אנחנו הופכים גיל שהוא מחרוזת לגיל שהוא מספר?  
נשתמש בפקודה:

```
int age = int.Parse(ageText);
```

אנו מציינים את הסוג (**int**), כותבים **Parse**. ומעבירים פנימה את **ageText** שלנו.  
כשנעשה את זה, נקבל מספר אמיתי ונוכל להדפיס אותו.  
אם נריץ את זה כעת ונקליד 43, הקומפיילר ידפיס **43**.  
עכשיו, כשנרצה לדעת מה יהיה הגיל בעוד 15 שנים ונכתוב:

```
Console.WriteLine(age + 15);
```

התוצאה תהיה 58.

**שימו לב** שזה כבר לא 4315, מכיוון שהתבצע כאן **Parse** (ניתוח/המרה).

המשמעות היא שה-**Parse** מקבל מחרוזת, בודק האם מדובר אך ורק במספר שעטוף בגרשיים, ואם כן, הוא מסיר את הגרשיים, מכניס את הערך הנקי לתוך המשתנה **age** ומסיים בהצלחה.

## הסכנה בקלט לא צפוי וקריסת התוכנה

מה קורה אם אנחנו מניחים שהמשתמש ייתן לנו מספר, אבל בפועל אסור לנו לעולם לסמוך על המשתמש? תמיד כשנעלה את התוכנה לסביבת Production, המשתמש יקליד משהו שלא צפינו לו, ולכן אנחנו חייבים לתכנן ולהתכונן לבלתי צפוי.

**לדוגמה**, אם המשתמש יכתוב את המספר במילים במקום בספרות, ויקליד **"Forty Three"**.

אנחנו נגיד שזה לא מספר.

למה זה לא מספר?

כי אם היינו כותבים מסמך ורוצים להתייחס למספר **43**, יכולנו להדפיס את המילים האלו כי הן מייצגות מספר, אבל מנקודת המבט של התוכנה שלנו, זה לא מספר.

המחרוזת הזו לא תגרום לתוכנה שלנו להיות מספיק אינטליגנטית כדי להבין שהתכווננו למספר **43**.

היא לא מסוגלת לעשות את זה.

כתוצאה מכך, אנחנו נקבל קריסה של האפליקציה, שנקראת **"Unhandled Exception"** (חריגה לא מטופלת).

זו לא חריגה רעה במובן של באג בקוד שלנו, אלא היא קורת כי קיבלנו מידע שגוי.

אנחנו לא צריכים לנסות להכריח את האפליקציה שלנו לעבוד עם מידע שגוי, כי אם נעשה את זה, אנחנו עלולים להכניס לאפליקציה שלנו דברים שפשוט לא עובדים.

אנחנו לא רוצים לשים ערכי ברירת מחדל או להעמיד פנים שזה עבד.

**אנחנו רוצים להגיד:** תעצור את התהליך, אל תמשיך, סיימנו.

## השיטה הבטוחה באמצעות `int.TryParse` ומשתנה `out`

כעת אנחנו צריכים לטפל בחריגה בצורה נכונה, כי אם המשתמש מעביר את המילים **"Forty Three"** או כל דבר מטורף אחר, אנחנו חייבים לדעת מה לעשות עם זה.

נשתמש בסוג שונה של פקודה:

```
int.TryParse(ageText, out int age);
```

**בעזרת הפקודה הזו אנחנו מנסים לגרום לזה לעבוד:** לנתח כמו שצריך ולהסיר את הגרשיים, ואם התהליך מצליח, אנחנו נכניס את הערך לתוך המשתנה **age**.

אבל אם זה נכשל, **שימו לב** שהפקודה **TryParse** מחזירה ערך **Boolean** (אמת/שקר), כך שאנחנו יכולים לכתוב:

```
bool isValidInt = int.TryParse(ageText, out int age);
```

עכשיו אנחנו יודעים האם מדובר במספר שלם (**Integer**) תקין.  
נוכל להדפיס האם המספר תקין (**false ix true**) ומה הערך של גיל.  
אם התוצאה היא **false**, נוכל לדעת זאת ולהתעלם מהמספר, אבל את ערך הגיל נדפיס בכל מקרה, אם הקלט לא תקין יודפס ערך ברירת המחדל (**0**), ואם הקלט היה מספר תקין, יודפס הערך המומר האמיתי.

## הגדרת משתנה inline לעומת שני שלבים

בפקודה שכתבנו, הגדרנו את המשתנה **int age** ישירות בתוך שורת הפקודה (**inline**) ומיד התחלנו להשתמש בו. הסיבה שעשינו את זה כך היא כדי לחסוך תהליך של שני שלבים, שבו קודם מצהירים על משתנה ואז מכניסים אליו ערך.  
**עם זאת, אפשר בהחלט לעשות את זה בשני שלבים:** קודם להצהיר על **int age** ובשורה הבאה לקרוא למתודת **TryParse** ולהעביר את **age** כמשתנה **out**.  
המשמעות של משתנה **out** היא שהמתודה גם תחזיר לנו ערך בוליאני (**האם ההמרה הצליחה**), וגם תפלוט ערך לתוך משתנה ה-**age** שלנו אם ההמרה תקינה.

## הבדל קריטי בין שרשור מחרוזות לחיבור מתמטי

כשקולטים מידע מהמשתמש באמצעות:

```
Console.ReadLine();
```

המידע תמיד מגיע כ-**string**, גם אם המשתמש הקליד ספרות כמו "43".  
**בשפת C#, סימן ה- (+) משמש לשני תפקידים שונים לחלוטין:** חיבור חשבוני בין מספרים, ושרשור (**הצמדה**) בין טקסטים.  
ברגע שהקומפיילר מזהה שאחד הצדדים של סימן הפלוס הוא טקסט (**כמו המשתנה ageText**), הוא אוטומטית ממיר גם את הצד השני לטקסט ומצמיד אותם זה לזה.  
לכן "43" ועוד 15 הופך לטקסט "4315" במקום לחישוב המתמטי 58.

## המרה ישירה בעזרת int.Parse והסכנה שבה

כדי לבצע חישוב מתמטי, חייבים להמיר את המחרוזת למספר שלם (**int**).  
**הפקודה int.Parse עושה פעולה ישירה:** היא לוקחת את המחרוזת ומנסה לחלץ ממנה מספר.  
אם המחרוזת מכילה אך ורק ספרות (**למשל "43"**), ההמרה תצליח.  
אבל בעולם האמיתי של פיתוח תוכנה, הכלל הראשון הוא שלעולם אין לסמוך על קלט המשתמש.  
אם המשתמש יקליד טקסט כמו "Forty Three", אותיות, או אפילו ישאיר שורה ריקה, הפקודה **int.Parse** לא תדע מה לעשות ותגרום לקריסה מיידית של התוכנה (**הודעת השגיאה Unhandled Exception**).  
קריסה כזו היא מצב לא מקצועי שאסור שיקרה במערכת אמיתית.

## הפתרון המקצועי והבטוח בעזרת int.TryParse

כדי למנוע קריסות, עובדים עם פקודה חכמה ובטוחה יותר שנקראת `int.TryParse` (נסה להמיר).

**פקודה זו לא קורסת לעולם, והיא עושה שני דברים במקביל:**

**הדבר הראשון** הוא בדיקה והחזרת ערך **בוליאני** - הפקודה מחזירה **true** אם ההמרה הצליחה והקלט אכן היה מספר, ומחזירה **false** אם הקלט היה משובש ולא חוקי.

**הדבר השני** הוא הוצאת הערך המומר החוצה - באמצעות המילה השמורה **out**, הפקודה שופכת את המספר הנקי לתוך משתנה עזר מיוחד (**במקרה שלנו, age**).

אם ההמרה נכשלה (**הוחזר false**), המשתנה **age** יקבל אוטומטית את ערך ברירת המחדל של מספרים שלמים, שהוא **0**, והתוכנה תמשיך לרוץ בצורה חלקה ובטוחה.

## כתיבה מקוצרת Inline לעומת כתיבה ארוכה בשני שלבים

שתי דרכים תחביריות להשתמש בפרמטר **out**.

הדרך הישנה והארוכה (**שני שלבים**) דורשת להצהיר על המשתנה **int age** בשורה נפרדת, ורק בשורה הבאה להעביר אותו לפקודה.

הדרך המודרנית והמועדפת (**Inline**) מאפשרת להצהיר על המשתנה **int age** ישירות בתוך הסוגריים של פקודת ה-**TryParse**.

זה הופך את הקוד לקצר יותר, קריא יותר וממוקד יותר, וזה הסטנדרט המקובל ביותר בתעשייה כיום.

~~~~~

בואו נריץ את זה: על המסך נראה את השאלה

```
"What's your age: "
```

נכתוב בצורה מספרית את המספר **43**, ועל המסך יודפס שההמרה תקינה (**True**), שהמספר הוא **43**, ושארנומנט הגיל ועוד **15** שווה ל-**58**.

זה עובד מצוין.

אבל מה קורה אם נריץ את זה שוב והפעם נכתוב במילים

```
"Forty Three"
```

זוהי אינה תצוגה מספרית תקינה של גיל (**ייצוג טקסטואלי של מספר הוא לא אותו דבר כמו מספר**).

על המסך יודפס שההמרה אינה תקינה (**False**), המספר שיתקבל הוא **0**, ופלט החיבור יציג את המספר **15**.

המספר שקיבלנו הוא **0**, מכיוון שזהו ערך ברירת המחדל של טיפוס מספר שלם (**Integer**).

שימו לב שקיבלנו בתוצאה הסופית את המספר **15**.

כאן אנחנו נכנסים לאזור אפור שבו כנראה לא נרצה לנהוג כך.

ללא משפט תנאי (**if**) שיבדוק אם ההמרה נכשלה ויעצור את התהליך, הקוד שלנו פשוט ממשיך לרוץ.

מאחר שהתייחסנו לגיל כאל **0** (ערך ברירת המחדל), החישוב של הגיל ועוד **15** נתן את התוצאה **15**, וזה כמובן שגוי לחלוטין.

כאן בדיוק טמון החלק המכשיל, שבו לפעמים שימוש בשיטה **Parse** הרגילה הוא האפשרות הנכונה ביותר, כי אנחנו רוצים לעצור את הריצה לחלוטין במקרה של שגיאה, ו-**Parse** בהחלט תעשה זאת אם נעביר לה מידע שגוי.

לכן, עלינו לשים לב לכך שהאפליקציה שלנו ממשיכה לרוץ ללא תלות בשאלה האם המשתנה

```
{isValidInt}
```

קיבל ערך תקין או לא, וממצב כזה אנחנו חייבים מאוד להיזהר.

הסכנה הלוגית שבשימוש בשיטת **TryParse** ללא בדיקה של תוצאת ההמרה לאחר מכן.

כאשר אנחנו משתמשים ב-**TryParse**, המערכת מנסה להמיר את מחרוזת הטקסט לטיפוס מספרי.

אם המשתמש מזין קלט שאינו ספרות טהורות, ההמרה נכשלה, אך בניגוד לשיטות אחרות התוכנית אינה קורסת ואינה עוצרת.

במקום זאת, השיטה מחזירה ערך **False** לתוך המשתנה הבוליאני, ומציבה באופן אוטומטי את ערך ברירת המחדל של טיפוס **int**, שהוא המספר **0**, בתוך משתנה הגיל שלנו.

הבעיה המרכזית היא מצב של כשל שקט במערכת.

אם לא נכתוב קוד מפורש שבודק האם המשתנה הבוליאני מחזיק בערך **True** ורק אז ממשיכים, התוכנית תמשיך לרוץ כרגיל כאילו לא קרה כלום.

היא תבצע חישובים מתמטיים ועיבוד נתונים על בסיס המספר **0** במקום הגיל האמיתי של המשתמש.

כתוצאה מכך מתקבלת תוצאה שגויה לחלוטין, מה שיוצר באגים לוגיים חמורים שקשה מאוד לאתר בסביבת הייצור.

ההשוואה לשיטת **Parse** הרגילה מדגישה מתי נכון להשתמש בכל כלי.

שיטת **Parse** מתאימה למצבים שבהם אנחנו מעדיפים שהתוכנית תעצור מיד ותזרוק שגיאה במקרה של קלט שגוי, כדי לא לאפשר לנתונים משובשים לקלקל את המערכת.

לעומת זאת, אם בוחרים להשתמש ב-**TryParse** כדי למנוע קריסה של האפליקציה, חובה תמיד לעטוף את המשך הלוגיקה במשפט תנאי שמוודא שההמרה באמת הצליחה לפני שמבצעים פעולות נוספות על המספר.

~~~~~

זה **TryParse**.

יש לנו **TryParse** למעשה על כל סוג משתנה.

ה- **Integer**, **Decimal**, **Double**, **Boolean** וכדומה.

לכן, קיים הרעיון שעלינו לבצע המרה (**Parse**) כדי לוודא שהערך אכן תקין, מכיוון שלפעמים הוא לא יהיה תקין.

## הצורך בבדיקת תקינות נתונים

בתעשיית הפיתוח, מידע שמגיע ממשתמשי קצה, מקבצים או ממערכות חיצוניות מתקבל לרוב כ-**String**, והוא לא תמיד תקין או תואם למה שהמערכת מצפה לראות.

אם ננסה להמיר טקסט שגוי למספר באמצעות המרה רגילה, התוכנה תקרוס ותזרוק שגיאת מערכת (**Exception**). אנחנו חייבים לוודא שהערך קביל ותקין לפני שאנחנו עובדים איתו, כדי למנוע קריסות ולהבטיח את יציבות האפליקציה.

## איך מנגנון TryParse פותר את הבעיה

המתודה **TryParse** היא השיטה הבטוחה, היעילה והתקנית ביותר ב-**C#** לבצע המרות טיפוסים. במקום לקרוס כאשר הערך אינו תקין, המתודה מנסה לבצע את ההמרה בצורה מבוקרת ומחזירה תשובה לוגית של **True** או **False** שמציינת האם ההמרה הצליחה. אם הערך תקין, נקבל **True** ואת הערך המומר מוכן לשימוש. אם הערך אינו תקין, נקבל **False** והתוכנה תמשיך לרוץ בצורה חלקה ללא קריסה, מה שמאפשר לנו לטפל בשגיאה בצורה אלגנטית ומקצועית.

## למה המנגנון קיים כמעט על כל סוגי המשתנים

מכיוון שהצורך בהמרה בטוחה של נתונים הוא קריטי בכל מערכת תוכנה יציבה, שפת **C#** מספקת את המתודה הזו עבור כל סוגי הנתונים הבסיסיים. בין אם מדובר במספרים שלמים, מספרים עשרוניים, ערכים לוגיים או תאריכים, המנגנון פועל בצורה זהה לחלוטין ומאפשר כתיבת קוד אחיד, נקי ועמיד בפני תקלות.

~~~~~

Cast

ישנה דרך נוספת להמיר מערך אחד לאחר:

```
double testDouble = age;
```

שורה זו תעבוד, מכיוון שאנחנו יכולים להכניס משתנה מסוג **Integer** (מספר שלם) לתוך **Double** (מספר עשרוני), מאחר שאיננו מאבדים מידע בתהליך.

אך אם נכתוב:

```
decimal testDecimal = testDouble;
```

השורה הזו לא תעבוד, בגלל העובדה שאנחנו מנסים להמיר מ-**Double** ל-**Decimal**.

כדי שזה יעבוד, נוכל לבצע פעולה שנקראת **Cast** (המרה מפורשת).

ה-Cast הזה בעצם יאמר למערכת: הערך הזה הוא כעת **Decimal**, וכך זה ייראה בקוד:

```
decimal testDecimal = (decimal)testDouble;
```

המשמעות היא שאנחנו הולכים להתייחס לערך הזה כאל **Decimal**, מאחר שזה עדיין מספר עם נקודה עשרונית; פשוט רמת הדיוק ביניהם היא שונה, ואנחנו אומרים לקומפיילר שלא אכפת לנו מההבדל הזה ואנחנו יכולים להכניס אותו לטיפוס **Decimal**.

לכן, נוכל להמיר מטיפוס אחד לאחר אם נבצע עליו **Cast**.

עם זאת, אי אפשר תמיד לעשות **Cast** לכל דבר.

ישנם דברים שאי אפשר להמיר בצורה כזו בגלל הלוגיקה והאופן שבו הטיפוסים עובדים, אבל במקרים שבהם זה אפשרי, תמיד נוכל לעשות זאת כך:

```
(decimal)
```

ולומר למערכת שאנחנו מודעים לכך ואנחנו מבצעים המרה שפשוט אומרת: כן, אנחנו רוצים שהערך יהיה מטיפוס אחר.

המרה מרומזת (Implicit Conversion) ללא איבוד מידע

כאשר מעבירים מספר שלם (**int**) לתוך משתנה שמכיל מספר עשרוני (**double**), הקומפיילר של **C#** מבצע את ההמרה באופן אוטומטי ושקוף לחלוטין.

הסיבה לכך היא שטיפוס **double** בעל טווח קיבולת רחב בהרבה ויכול להכיל בתוכו כל מספר שלם.

לכן, מובטח לנו מבחינה ארכיטקטונית שלא יאבד שום מידע או דיוק במספר במהלך ההעברה.

למה אי אפשר להעביר ישירות מ-Double ל-Decimal?

למרות שגם **Double** וגם **Decimal** הם טיפוסים של מספרים עשרוניים, הם מיוצגים ומחושבים בצורה שונה לחלוטין בזיכרון המחשב.

טיפוס **Double** עובד בשיטה מהירה של נקודה צפה שמתאימה לחישובים כלליים ומדעיים, בעוד שטיפוס **Decimal** עובד בשיטה מדויקת במיוחד המיועדת לחישובים פיננסיים וכספיים.

בגלל ההבדל ברמת הדיוק ובאופן הייצוג בזיכרון, העברה ישירה מ-**Double** ל-**Decimal** עלולה לגרום לעיוותים קלים או לחיתוך של המידע.

לכן, הקומפיילר עוצר אותנו בכוונה ולא מאפשר המרה אוטומטית, כדי להגן על הקוד מבאגים שקטים של איבוד נתונים.

הפתרון: שימוש ב-Cast (המרה מפורשת - Explicit Conversion)

כדי לעקוף את החסימה של הקומפיילר, אנחנו משתמשים בטכניקה שנקראת **Cast**.
כשאנחנו מוסיפים את שם הטיפוס בסוגריים לפני המשתנה, במבנה של

```
(decimal)testDouble
```

אנחנו בעצם נותנים פקודה ישירה לקומפיילר ואומרים לו: אנחנו המפתחים ואנחנו מודעים להבדלים ברמת הדיוק בין הטיפוסים; קח את הערך הקיים מתוך ה-**Double** ותאלץ אותו להפוך ל-**Decimal** תחת האחריות שלנו .

הגבולות והמגבלות של פעולת ה-Cast

אי אפשר להשתמש ב-**Cast** על כל שני טיפוסים באופן עיוור.

כדי שהמרה מפורשת כזו תצליח, חייב להיות קשר לוגי ומתמטי מוכר בין הטיפוסים ברמת שפת **#C**.

למשל, בין טיפוסים מספריים שונים (כמו **int**, **double**, **float**, **decimal**) ה-**Cast** יעבוד מצוין.

לעומת זאת, אם ננסה לעשות **Cast** מטיפוס של **string** או מטיפוס **Boolean** לטיפוס מספרי, הקומפיילר לא יאפשר זאת או שהתוכנית תקרוס בזמן ריצה, מכיוון שהלוגיקה של הזיכרון ביניהם אינה תואמת כלל.